

拓扑 AC CSP-S & NOIP 线上联测第二轮第一场讲评

袁胜利

2026 年 8 月 10 日

吐槽环节/自由分享

不交叉配对

两排字符只能把相同字母连成一对，每个位置至多使用一次，并且连线不能交叉。求最多能连多少对。

$n \leq 10^3$, $m \leq 10^6$ 。

不交叉配对

子任务一：这是经典 LCS。令 $f_{i,j}$ 表示两个前缀的最长公共子序列长度，直接转移并滚动数组，复杂度 $O(nm)$ 。

不交叉配对

子任务一：这是经典 LCS。令 $f_{i,j}$ 表示两个前缀的最长公共子序列长度，直接转移并滚动数组，复杂度 $O(nm)$ 。

子任务二：若 S_1 全由字符 c 组成，答案就是 $\min(n, \text{count}_{S_2}(c))$ ，复杂度 $O(n + m)$ 。

不交叉配对

连线不交叉，等价于求两个字符串的 LCS。普通 DP 的问题是第二维有 10^6 个位置。

对 S_2 的每种字符，保存它出现的所有位置。令 f_k 表示：处理完 S_1 的当前前缀，选出长度为 k 的公共子序列时，在 S_2 中最小的结束位置。结束得越早，后面越容易继续匹配，所以只保留最小值即可。

读到字符 c 时，不选它就保留 f_k ；选它就从旧的 f_{k-1} 出发，在 c 的位置表中二分第一个严格更大的位置：

$$g_k = \min(f_k, \text{next}(c, f_{k-1})).$$

初始 $f_0 = -1$ ，最后最大的可达 k 就是答案。复杂度 $O(m + n^2 \log m)$ 。

二进制生长

每个数都可以反复变成 $2x$ 或 $2x+1$ 。分别操作若干次后，最小化最终数组的最大值与最小值之差。

$n \leq 2 \times 10^5$, $a_i \leq 10^{18}$ 。

二进制生长

子任务一：记最大二进制位长为 B 。每个数只需考虑扩展到 $B-1, B, B+1$ 位，枚举每个数选择哪个可达区间，再计算这些区间的最短覆盖，复杂度 $O(n3^n)$ 。

二进制生长

子任务一：记最大二进制位长为 B 。每个数只需考虑扩展到 $B-1, B, B+1$ 位，枚举每个数选择哪个可达区间，再计算这些区间的最短覆盖，复杂度 $O(n3^n)$ 。

子任务二：此时 $B \leq 9$ 。枚举答案区间左端点 L ；对每个数求不小于 L 的最小可达值，取这些值的最大值作为 R ，复杂度 $O(n2^B)$ 。

二进制生长

子任务一：记最大二进制位长为 B 。每个数只需考虑扩展到 $B-1, B, B+1$ 位，枚举每个数选择哪个可达区间，再计算这些区间的最短覆盖，复杂度 $O(n3^n)$ 。

子任务二：此时 $B \leq 9$ 。枚举答案区间左端点 L ；对每个数求不小于 L 的最小可达值，取这些值的最大值作为 R ，复杂度 $O(n2^B)$ 。

子任务三：此时端点值域不超过 $2^{B+1} \leq 2^{18}$ 。把区间出现、消失事件直接挂在值域上，用双指针维护覆盖了多少个下标，复杂度 $O(n + 2^B)$ 。

二进制生长

子任务一：记最大二进制位长为 B 。每个数只需考虑扩展到 $B-1, B, B+1$ 位，枚举每个数选择哪个可达区间，再计算这些区间的最短覆盖，复杂度 $O(n3^n)$ 。

子任务二：此时 $B \leq 9$ 。枚举答案区间左端点 L ；对每个数求不小于 L 的最小可达值，取这些值的最大值作为 R ，复杂度 $O(n2^B)$ 。

子任务三：此时端点值域不超过 $2^{B+1} \leq 2^{18}$ 。把区间出现、消失事件直接挂在值域上，用双指针维护覆盖了多少个下标，复杂度 $O(n + 2^B)$ 。

子任务四：枚举某个可达区间的右端点作为 L ，扫描所有可达区间求最小的 R ，复杂度 $O(n^2)$ 。

观察

在二进制下, $2x$ 和 $2x+1$ 分别是在末尾添加一位 0 或 1。
若 x 有 b 位, 把它扩展到 L 位后, 可达值恰好是一段连续区间:

$$\left[x2^{L-b}, (x+1)2^{L-b} - 1 \right].$$

设输入中的最大位长为 B 。最优答案中, 各数的最终位长至多相差一位; 否则把位长最小的数再扩展一位, 极差会缩小。另一方面, 若所有数都至少有 $B+2$ 位, 同时删去最后一位会得到更小的极差。
所以每个 $a_i > 1$ 只需考虑扩展到 $B-1, B, B+1$ 位, 产生至多三个可达区间。 $a_i = 1$ 可以变成任意正整数, 不会限制答案区间。

二进制生长

把同一个下标产生的区间看成同一种颜色。问题变成：求最短的整数区间 $[L, R]$ ，使它与每种颜色的至少一个区间相交。

最优的 L 一定可以右移到某个区间的右端点，最优的 R 一定可以左移到某个区间的左端点。因此分别排序所有左端点和右端点。

按右端点从小到大枚举 L 。维护所有右端点不小于 L 的区间，再让指针 R 扫过左端点并加入颜色，直到所有颜色都出现；此时用 $\max(0, R - L)$ 更新答案。

计算完当前 L 后，再删除右端点等于 L 的所有区间。因为区间是闭区间，相同右端点必须成组处理。复杂度 $O(n \log n)$ 。

异或替换

一次操作选择位置 u ，把 a_u 改成当前整个序列的异或和。保证可以从 a 变成 b ，求最少操作次数。

$n \leq 2 \times 10^5$ 。

异或替换

子任务一：注意到序列总可能性为 $O(n!)$ 种，因此直接搜索即可。

异或替换

- 子任务一：注意到序列总可能性为 $O(n!)$ 种，因此直接搜索即可。
- 子任务二：扩充后的初始序列是 $0, 1, \dots, n$ 的一个排列。分解目标置换；包含额外位置的长度 L 环需要 $L - 1$ 次交换，其他环需要 $L + 1$ 次，复杂度 $O(n \log n)$ 。

观察

把当前整个序列的异或和记作手中数 x 。操作位置 u 后，这个位置从 a_u 变成 x ，新的异或和为

$$x \oplus a_u \oplus x = a_u.$$

也就是说，一次操作正好把手中的 x 放到位置 u ，再把原来的 a_u 拿到手中。

因此可以在序列末尾增加一个额外位置：初始放 $x = \text{xor}(a)$ ，目标放 $y = \text{xor}(b)$ 。题目操作就变成额外位置与任意普通位置交换。

异或替换

只考虑 $a_i \neq b_i$ 的位置。建立一条从 b_i 指向 a_i 的边：手中为 b_i 时操作位置 i ，就能把它改正确，同时手中变成 a_i 。

忽略方向，加入顶点 x, y 。设失配边数为 m ，连通块数为 c 。可达性保证扩充后的初、末多重集合相同，因此每个连通块中的有向边可以排成欧拉迹或欧拉回路。

手中数一旦进入某个连通块，就能沿欧拉序依次修好其中每条失配边。初始手中数已经位于 x 所在的连通块；其余每个连通块都要多操作一次进入并返回。

所以 $m > 0$ 时答案为 $m + c - 1$ ；若 $m = 0$ ，答案为 0。排序离散化后用并查集统计 c ，复杂度 $O(n \log n)$ 。

沿途密钥

树上有若干份档案。档案 i 的密钥在 x_i ，档案在 y_i ，价值为 w_i 。选择一条有方向的简单路径；只有先经过 x_i 、再经过 y_i ，这份档案才产生贡献。 $n, m \leq 10^5$ ，价值可以为负数。

沿途密钥

子任务一：枚举有序端点 (s, t) ，再枚举档案，用树上距离判断路径是否依次经过 x_i, y_i 。规模很小，直接实现即可。

沿途密钥

子任务一：枚举有序端点 (s, t) ，再枚举档案，用树上距离判断路径是否依次经过 x_i, y_i 。规模很小，直接实现即可。

子任务二：枚举起点 s ，从 s 做一次 DFS。递归栈就是当前路径，进入、退出结点时维护已经取得的密钥和档案价值，复杂度 $O(n(n + m))$ 。

沿途密钥

子任务一：枚举有序端点 (s, t) ，再枚举档案，用树上距离判断路径是否依次经过 x_i, y_i 。规模很小，直接实现即可。

子任务二：枚举起点 s ，从 s 做一次 DFS。递归栈就是当前路径，进入、退出结点时维护已经取得的密钥和档案价值，复杂度 $O(n(n+m))$ 。

子任务三：最优路径的起点一定为某个 x_i 。枚举至多 m 个关键点作为起点进行 DFS，时间复杂度 $O(nm)$ 。

子任务四：此时只要路径经过点 v ，聚合在 v 的价值就会计入。问题变成树上最大权简单路径，树形 DP 维护每点向下的最大链，复杂度 $O(n + m)$ 。

沿途密钥

子任务四：此时只要路径经过点 v ，聚合在 v 的价值就会计入。问题变成树上最大权简单路径，树形 DP 维护每点向下的最大链，复杂度 $O(n + m)$ 。

子任务五：所有密钥都在根 1，可以把最优路径缩成从根出发。把价值聚合到 y_i ，DFS 维护根到当前点的路径和，复杂度 $O(n + m)$ 。

沿途密钥

子任务四：此时只要路径经过点 v ，聚合在 v 的价值就会计入。问题变成树上最大权简单路径，树形 DP 维护每点向下的最大链，复杂度 $O(n + m)$ 。

子任务五：所有密钥都在根 1，可以把最优路径缩成从根出发。把价值聚合到 y_i ，DFS 维护根到当前点的路径和，复杂度 $O(n + m)$ 。

子任务六：把链编号后分左右两个方向。对从左向右的行程，档案贡献等价于 $L \leq X \leq Y \leq R$ ；扫描 L ，线段树对合法的 R 区间加，复杂度 $O((n + m) \log n)$ 。

观察

一条有向路径由有序端点 (s, t) 唯一确定。先考虑一份满足 $x \neq y$ 的档案。

删去 x 通往 y 的第一条边，记包含 x 的连通块为 C_x ；再删去 y 通往 x 的第一条边，记包含 y 的连通块为 C_y 。

则路径 $s \rightarrow t$ 依次经过 x, y ，当且仅当

$$s \in C_x, \quad t \in C_y.$$

因为从 C_x 走向树的另一侧必先经过 x ，而要进入 C_y 又必须经过 y 。所以这份档案会给所有端点对 $C_x \times C_y$ 加上 w 。

沿途密钥

把树根定为 1，一棵子树在 DFS 序中是连续区间。

若 x, y 互不为祖先，则 C_x, C_y 分别是 x, y 的子树。若 x 是 y 的祖先，设 c 是 x 通往 y 的第一个儿子，则

$$C_x = V \setminus \text{subtree}(c), \quad C_y = \text{subtree}(y).$$

y 是 x 的祖先时同理。子树补集可以拆成两个 DFS 区间，所以 $C_x \times C_y$ 至多拆成两个“起点区间 $\times$ 终点区间”的矩形。祖先关系和儿子 c 用倍增求出。

还要单独处理 $x = y = v$ 。此时档案产生贡献，当且仅当路径经过 v 。
先给下面两个端点集合加上 w ：

$$(V \setminus \text{subtree}(v)) \times \text{subtree}(v), \quad \text{subtree}(v) \times V.$$

它们只会多算一种情况：两个端点都在 v 的同一个儿子子树中。对每个儿子 c ，再给

$$\text{subtree}(c) \times \text{subtree}(c)$$

减去 w 即可。先把同一个 v 上的价值聚合，所有修正矩形的总数仍是 $O(n + m)$ 。

沿途密钥

一个矩形 $[l_s, r_s] \times [l_t, r_t]$ 表示：起点在第一段 DFS 区间、终点在第二段 DFS 区间时，需要加上权值 w 。

扫描起点。在 l_s 处对终点区间 $[l_t, r_t]$ 加 w ，在 $r_s + 1$ 处减 w 。线段树的每个叶子表示一个终点，维护当前总价值；根结点就是当前起点对应的最优终点。

扫完所有起点后，根结点出现过的最大值就是答案。负权档案也直接做区间加，因为经过它时不能选择跳过。

矩形和事件总数为 $O(n + m)$ ，总复杂度 $O((n + m) \log n)$ 。